# Ders 9 Çalışma Özeti

## Ders 9 Çalışma Özeti

### Genel Konular

- Mediator (davranışsal kalıp, tasarım kalıplarının sonuncusu)
  - Amaç: bir nesne grubunun arasındaki etkileşimleri sarmalamak. Nesneler birbirleriyle doğrudan değil, dolaylı olarak (bir ara bulucu üzerinden) iletişim kurar.
  - Kazanç: nesneler arasındaki ilişkiler değiştiğinde bu nesnelerin kodunu değiştirmeden değişikliği yapabilmek.
  - Örnek (ekrandan, 00:10:00): font seçme penceresi (Font Chooser). Aile listesi (`aListBox`), giriş alanı (`anEntryField`), düğme (`aButton`), Weight (medium/bold/demibold), Slant (roman/italic/oblique), Size ve condensed seçenekleri vardır. Her değişiklik en üstteki önizleme yazısına ("The quick brown fox…") anında yansımalıdır. Bileşenlerin her biri `director` alanıyla `aFontDialogDirector` nesnesine bağlıdır.
  - Aynı radyo düğmesi grubundaki seçenekler (medium ve bold, roman ve italic) birbirini de etkiler; bileşenler hem birbirleriyle hem önizleme alanıyla etkileşir.
  - Hoca burada Observer kullanmanın anlamlı olmadığını söyledi; yapılacak işler daha anlık ve karşılıklıdır.
  - Bağlantı sayısı: 4 bileşen birbiriyle çift yönlü haberleşirse $4 \cdot 3 = 12$ bağlantı olur. 5. bileşen eklenince önceki eleman sayısı kadar, iki yönlü olduğu için $4 \cdot 2 = 8$ bağlantı daha gelir. Genel olarak $n$ bileşende $n(n-1)$ yönlü bağlantı vardır.
  - Çözüm: bileşenlerin birbirleriyle değil, merkezi bir bileşenle haberleşmesi. Bu merkezi bileşen iletişimleri sarmalar ve yönlendirmeleri yapar.
  - GUI çatılarında bileşenler zaten `Widget` soyut sınıfının alt sınıflarıdır; diyalog kutusu için soyut sınıf ve "bileşen değişti" bildirimi alan metot da hazırdır. Kendi `FontDialogDirector` sınıfımızı kalıtımla oluştururuz.
  - Kalıp yapısı (ekrandan, 00:14:30): soyut `Mediator` ve soyut `Colleague`; `Colleague` üst sınıfında `mediator` alanı vardır. `ConcreteMediator`, `ConcreteColleague1` ve `ConcreteColleague2`'ye bağlıdır. Colleague (meslektaş, peer) gerçeklemeleri kendi aralarında doğrudan değil, Mediator üzerinden haberleşir.
  - `ConcreteMediator` somut colleague nesnelerinin bir koleksiyonunu tutar, onların durumunu alır ve değiştirir. Colleague'ler `ConcreteMediator`'ı soyut üst sınıftaki `mediator` üye alanı sayesinde dolaylı olarak görür.
  - Haberleşme tek yönlü ya da çift yönlü olabilir; kalıp ikisini de destekler.
  - Zayıf yönü: karmaşıklığı halının altına süpürmek gibidir. Mediator gerçeklemesi çok özelleşmiştir, yeniden kullanımı zordur; her işbirliği senaryosu için sıfırdan kodlanır. Mediator sınıfı ilerde bir "large class" olur.
  - Ne zaman yararlı: bir grup nesne arasında iyi tanımlanmış (az değişecek) ama karmaşık etkileşimler varsa. Kaynak kodu elimizde olmayan sınıflarda ya da güncelleme dağıtırken yalnız Mediator'ı göndermek yeterli olur.
- Refactoring'e giriş
  - Tanım (ekrandan, 00:20:00): bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmek; yazılımın dışarıdan görünen davranışını (dışarıya açtığı kullanım biçimini, API'yi) değiştirmeden iç yapısını değiştirmek.
  - Yeni gereksinim eklemek ya da gereksinim değişikliğini kodlamak refactoring değildir. Amaç kodun iç kalite özelliklerini iyileştirmektir: anlaşılabilirlik, esneklik, yeniden kullanılabilirlik, yüksek uyum ve düşük bağımlılık (ekrandan, 00:20:00).
  - Bir üye alanın ya da metodun adını daha anlaşılır yapmak bile refactoring'dir.
  - Nasıl: çevik yaklaşımla, küçük adımlarla ve sürekli sınama eşliğinde. İlk tasarım mü-

kemmel olmak zorunda değildir; yanlış bir karar fark edildiğinde refactoring yapılır, ardından kod yeniden sınanır.
  - Ne zaman: kodun toparlanmaya ihtiyacı olduğu hissedildiğinde, yani yeni özellik eklemek, hata bulmak ve düzeltmek zorlaşmaya başladığında.
- Kod kusuru (code smell)
  - Kodun iyileştirilmesi gerektiğinin işaretidir; koddaki hastalıklardır; kullanılan programlama yaklaşımının kusurlu (suboptimal) kullanım örnekleridir. Kod işini yapar ama kusurlu yapar.
  - Yazılım zamanla, farklı kişilerin farklı zamanlarda yaptığı küçük değişikliklerle "yamalı bohçaya" döner. Bakım masrafları tavana vurmadan bu işaretlere bakılmalıdır.
  - Bir refactoring eylemi yeni bir kod kusuru doğurabilir; o da yeniden refactoring ile düzeltilir.
  - Ders Martin Fowler'ın "Refactoring" kitabından seçilmiş bir kümeyi işler. Kod kusurları ile refactoring eylemleri arasında çoka çok ilişki vardır.
- Birim sınaması ve JUnit
  - Birim sınaması en küçük yazılım bileşeninin sınanmasıdır; nesneye dayalı programlamada bireysel sınıfların sınanmasıdır (ekrandan, 00:30:00).
  - Ne zaman tasarlanır: kodlamadan önce (Test Driven Development), kodlama sırasında ya da kodlamanın ardından. Hatalar ne kadar erken bulunursa düzeltme maliyeti o kadar düşük olur.
  - Bir sınıfın ihtiyaç duyduğu diğer sınıflar henüz yoksa onların yerine geçen kod yazılır: vekil, sahte, yalancı kod/sınıf (stub, dummy, surrogate, proxy). Bunlar yalnız derleme hatası vermeyecek kadar iskelet koddur (ör. doğrudan `return true;` ya da sabit bir sayı döndürmek) ve gerçek sınıflar gerçeklenene dek kullanılır.
  - JUnit, Java kodunun birim sınaması için bir çerçeve programdır (xUnit ailesi); C# için csUnit gibi sürümleri vardır. Eclipse'de "build path → add libraries" ile eklenir (ekrandan, 00:30:00).
  - JUnit 3.x (ekrandan, 00:40:00): sınama sınıfı `junit.framework.TestCase`'ten kalıtımla türetilir; sınama metotlarının adı `test` ile başlar (`testXXX()`). Her `testXXX()` metodu bir test case'tir; içinde `assert...` çağrıları yapılır (tercihen bir metotta bir assert). `Assert` sınıfında `assertEquals(beklenen, sonuc)`, `assertNotEquals`, `assertTrue`, `assertFalse`, `assertNull`, `assertNotNull` vardır; hepsi statiktir. `setUp()` (protected) her `testXXX()` öncesinde yeniden çalışır ve sınama ortamını sıfırdan kurar. `TestAll.suite()` statik metodu ile sınama sınıfları bir test suite altında toplanabilir.
  - JUnit 4.x (ekrandan, 00:42:30): annotation desteği gelir. `TestCase`'ten türetmek gerekmez; `import static org.junit.Assert.*;` ve `import org.junit.*;` yazılır. Metotlara `@Test`, hazırlık metoduna `@Before` konur. Beklenen exception `@Test(expected=SomeException.class)` ile sınanır. JUnit 3'ün en önemli eksiği exception sınamasıdır.
  - Annotation kaynak kod üzerinde çalışan başka araçlara verilen işarettir; ör. `@SuppressWarnings("...")` derleyiciye belirli uyarıları göstermemesini söyler.
- Video kiralama örneği (Fowler'ın kitabındaki örnek)
  - Sınıflar (ekrandan, 00:50:00): `Movie` (`- _priceCode : int`), `Rental` (`- _daysRented : int`), `Customer` (`+ statement()`). Customer ile Rental arasında bire çok, Rental ile Movie arasında bire bir ilişki vardır.
  - Rental sınıfı kira işlemini ayrı bir problem bileşeni olarak modeller; Customer ile Movie doğrudan bağlansaydı film başka müşteriye kiralanınca eski kira bilgisi kaybolurdu.
  - `statement()` her kira için ücret ve bonus puanı hesaplar, makbuz satırını basar, en sonda toplam borcu ve puanı yazar (ekrandan, 00:50:00, sıralama şeması).
  - İş kuralları (ekrandan, 00:57:40): normal film 2.0, 2 günü geçen her gün için +1.5; yeni film günlük 3.0; çocuk filmi 1.5, 3 günü geçen her gün için +1.5. Her kira 1 puan, iki günlük yeni film kirası için 1 ek puan.
  - Fiyat türleri `public static final int` sabitleriyle tanımlanmıştır: `CHILDRENS = 2`, `REGULAR = 0`, `NEW_RELEASE = 1` (ekrandan, 01:00:00). Hoca enum bile kullanılmadığını eleştirdi.
  - Üye alanların `_` ile başlaması, siyah beyaz basılan kitapta üye alanı parametreden ayırt etmek içindir; Java'da önerilen bir kural değildir.
  - `statement()` içindeki `switch` ilk eleştiridir: switch-case gördüğünüz yerde kalıtım ve çok biçimlilik gerekir.

- Örnek üzerinde JUnit
  - Üretim kodu ve sınama kodu ayrı klasörlerde ama aynı pakette (`fowler_00`) tutulur.
  - `TestMovie`: farklı türdeki filmlerin fiyat kodlarının eşit olmadığını `assertTrue(mesaj, ...)` ve `assertFalse` ile sınar. Her assert metodunun başına başarısızlıkta gösterilen bir `String` mesaj alan aşırı yüklenmiş sürümü vardır.
  - `TestCustomer` (ekrandan, 01:10:00 ve 01:11:45): `setUp()` bir müşteri ve dört film oluşturur. Testler: `testGetName`, `testStatementWhenEmpty`, `testStatementWithMoviesShortRent`, `testStatementWithMoviesLongRent`, `testNewReleaseRentalBonus`. Beklenen makbuz metni elle yazılır ve `assertEquals(beklenen, sonuc)` ile karşılaştırılır.
  - Bonusu ayrıca sınamak istense de tasarım buna izin vermez; bonus yalnız makbuzun tamamı üzerinden sınanabilir.
  - Hata canlandırması (ekrandan, 01:11:45): çocuk filmi ücreti bilerek 1.0 yazılınca kırmızı çubuk çıkar; `ComparisonFailure` beklenen "Monsters, Inc. 1.[5]" ve "Amount owed is 6.5" yerine "1.[0]" ve "6.0" gösterir. Değer 1.5'e döndürülünce çubuk yeşile döner.
  - `TestAll` (ekrandan, 01:13:20): `public static Test suite()` içinde `new TestSuite("Bütün testler")`, `addTestSuite(TestCustomer.class)` ve `addTestSuite(TestMovie.class)`; toplam 6 test çalışır. `.class` Java'nın Reflection API'sinin parçasıdır.
- Kod kusurları ve refactoring eylemleri
  - Duplicated Code (yinelenen kod): aynı ya da çok benzer kodun tekrarı. Değişiklik gerektiğinde her kopyayı ayrı ayrı değiştirmek gerekir.
  - Extract Method (metot çıkartma, ekrandan, 01:40:00): metot içindeki kodun bir kısmını yeni bir metot yapmak. Birçok kod kusurunu düzeltir. Yeni metoda amacını açıkça belirten bir ad verilir.
  - Long Method (uzun metot): anlaşılmayı zorlaştırır, gereksinim değişikliklerinden etkilenme olasılığını artırır, uyumu düşürür. Çözüm yine Extract Method; ayrıca Decompose Conditional, Replace Temp with Query ve Replace Method with Method Object.
  - Pull Up Method (ekrandan, 01:50:00 ve 01:54:40): yinelenen kod kalıtım ağacındaki kardeş ya da kuzen sınıflardaysa çıkarılan metot ortak üst sınıfa çekilir. Kodlar çok farklıysa en azından metot üst sınıfta soyut tanımlanır.
  - Form Template Method: kod aynı değil ama benzerse GoF Template Method kalıbı uygulanır.
  - İlişkisiz sınıflarda yinelenen kod: sorumluluklar yanlış dağıtılmış olabilir; metot bir sınıfa ya da yeni bir sınıfa konur, diğerleri onu kullanır.
  - Decompose Conditional (karar verme komutunu parçalama, ekrandan, 02:00:00): karmaşık `if` koşulu ve dalları anlamlı adlı metotlara çıkarılır (`notSummer`, `summerCharge`, `winterCharge`).
  - Long Parameter List: birlikte gezen parametreler bir varlığın durum bilgisinin parçasıdır. Çözüm Introduce Parameter Object ya da Preserve Whole Object.
  - Introduce Parameter Object (ekrandan, 02:06:40): `start` ve `end` parametreleri `DateRange` sınıfında toplanır; bu sınıf `isWithinRange(Date)` gibi metotların da doğal yeridir.
  - Preserve Whole Object (tüm nesneyi gönder, ekrandan, 02:10:00): `low` ve `high` yerine `daysTempRange()` nesnesinin kendisi gönderilir. Zayıf yönleri: bağımlılık artabilir; metot imzası değiştiği için geriye doğru uyumluluk kaybedilir.
  - Replace Temp with Query (geçici değişken yerine erişim metodu, ekrandan, 02:16:40 ve 02:20:00): geçici değişken yerine bir sorgu metodu yazılır (`basePrice()`, `discountFactor()`).
  - Replace Method with Method Object (ekrandan, 02:27:20): uzun ve geçici değişkenleri karışık bir metot yeni bir sınıfa (`PriceCalculator`) taşınır; yerel değişkenler üye alan olur, gövde `compute()` metoduna gider.

## Hocanın Özellikle Vurguladığı Kısımlar

- Mediator kalıbı bir sihir değildir
  - Nesne sınıflarının kodu değişmez ama karmaşıklık tek bir Mediator sınıfında toplanır. Yeniden kullanımı zordur; etkileşimler iyi tanımlıysa kullanılmalıdır.
- Refactoring'in iki bileşeni

  - Ne yapıldığı değişmez, nasıl yapıldığı değişir; dışarıya açılan kullanım biçimi korunur. Yeni özellik eklemek refactoring değildir.
- Refactoring sınama ile birlikte yapılır
  - Her küçük adımdan sonra birim sınamaları yeniden koşulur; sınama kodu ayrı klasörde ama aynı pakette tutulur.
- Sınama metotlarına uzun da olsa açık adlar verilmeli
  - `testStatementWithMoviesLongRent` gibi neyin sınandığını anlatan adlar tercih edilir.
- Beklenen sonuç elle yazılmalı
  - Beklenen makbuz metni kodla üretilirse hata keşfedilemez.
- Kod öyle yazılmalı ki yoruma gerek kalmasın
  - Extract Method ile çıkarılan metotların adları yorumların yerini tutar.
- Metot çağrısı maliyeti kaygısı
  - Modern dillerde bağlam değiştirme (context switching) yükü artık hissedilmez; böl, parçala, yönet mantığı çekinmeden kullanılabilir.
- Darboğaz ilkesi
  - Replace Temp with Query sonrası fazladan çağrılar çoğu zaman sistemin asıl darboğazının yanında küçük kalır. Kuşkuda profiler ile ölçülür; başarım düşmüşse küçük adım geri alınır. Dağıtık API çağrılarında bu refactoring yapılmaz.
- Geçici değişkenlere "gıcıklık"
  - Geçici değişkenler metodu uzatmaya ve kopyala yapıştıra iter; kümülatif toplam gibi zorunlu durumlar dışında azaltılmalıdır.
- Preserve Whole Object kararı
  - Kazançlar ve kayıplar (bağımlılık, geriye doğru uyumluluk) terazinin iki kefesine konup karar verilir.

## Kısa Tekrar Notları

- Mediator: colleague nesneleri birbirleriyle değil, `ConcreteMediator` üzerinden haberleşir; $n$ bileşende $n(n-1)$ doğrudan yönlü bağlantının yerine $n$ bileşen-merkez bağlantısı gelir.
- Mediator'ın bedeli: özelleşmiş, yeniden kullanılamayan, büyüyen bir merkez sınıf.
- Refactoring = davranışı ve dış arayüzü koruyarak iç yapıyı iyileştirmek.
- Code smell = kodun kusurlu (suboptimal) olduğunun işareti.
- Kod kusurları ile refactoring eylemleri arasında çoka çok ilişki vardır.
- Birim sınaması: NYP'de bireysel sınıfın sınanması; TDD'de kod yazılmadan önce tasarlanır.
- Stub/dummy/surrogate/proxy: henüz yazılmamış sınıfların yerine geçen iskelet kod.
- JUnit 3: `extends TestCase`, `testXXX()`, `setUp()`, statik `assert...`, `suite()`.
- JUnit 4: `@Test`, `@Before`, `@Test(expected=...class)`, `TestCase`'ten türetme gerekmez.
- Video kiralama: `Customer` 1—* `Rental` *—1 `Movie`; `statement()` uzun ve `switch` içerir.
- Duplicated Code → Extract Method, Pull Up Method, Form Template Method.
- Long Method → Extract Method, Decompose Conditional, Replace Temp with Query, Replace Method with Method Object.
- Long Parameter List → Introduce Parameter Object, Preserve Whole Object.

## Detaylı Açıklamalar

- Mediator neden Observer değil?
  - Font penceresinde bileşenler birbirini ve önizlemeyi anlık ve karşılıklı etkiler. Hoca bu nedenle bildirim-abone ilişkisinin değil, etkileşimi tek yerde toplayan bir merkezin uygun olduğunu söyledi. Merkez nesne (`FontDialogDirector`) bir bileşen değiştiğinde hangi bileşenin nasıl güncelleneceğine karar verir.
- Mediator ve dağıtım
  - Kalıbın pratik getirisi: davranış değiştiğinde müşteriye yalnız Mediator sınıfının güncellemesini göndermek yeterlidir; daha önce dağıtılmış bileşen sınıfları değişmez.
- Refactoring bir yatırımdır
  - Zaten doğru çalışan kodda yapılır; amaç, sonraki gereksinim değişikliklerinde ve sınamalarda harcanacak çabayı azaltmaktır. Yanlış bir tasarım kararı fark edildiğinde küçük adımlarla düzeltilir ve her adımın ardından sınama yapılır.
- JUnit 3 ile `TestCustomer` sınaması

  - `setUp()` her test öncesi müşteriyi ve dört filmi yeniden oluşturur; böylece her test temiz bir durumla başlar. `testNewReleaseRentalBonus` (ekrandan, 01:10:00) iki kira ekler: `Surrogates` 2 gün ve `Terminator Salvation` 1 gün. Beklenen makbuz:
    ```
    Rental Record for <müşteri adı>
        Surrogates  6.0
        Terminator Salvation    3.0
    Amount owed is 9.0
    You earned 3 frequent renter points
    ```
  - Yeni film günlük 3.0 olduğundan $2 \cdot 3.0 = 6.0$ ve $1 \cdot 3.0 = 3.0$, toplam $9.0$. Puan: her kira 1 puan (2 puan) ve 2 günlük yeni film kirası için 1 ek puan, toplam 3.
  - Ders sonrası not: Hoca ek puanı "yeni film iki günden uzun kiralanırsa" diye anlattı. Ekrandaki test 2 günlük kirada ek puan beklediği için koddaki koşul büyük olasılıkla "1 günden fazla" (`> 1`) biçimindedir; koddaki yorum da "two day new release rental" der (ekrandan, 01:11:45).
- Hata ayıklama örneği
  - Çocuk filmi taban ücreti 1.5 yerine 1.0 yazılınca kısa ve uzun kira testleri başarısız olur. `ComparisonFailure` beklenen ve elde edilen metnin farklılaştığı ilk ve son yeri köşeli parantezle işaretler. Kısa kira testinde beklenen toplam 6.5 iken 6.0 elde edilir; fark tam olarak $1.5 - 1.0 = 0.5$'tir.
- Extract Method ve değişkenler (ekrandan, 01:42:10)
  - `printOwing()` içinde başlık basma, `outstanding` hesaplama ve ayrıntı basma işleri `printBanner()`, `getOutstanding()` ve `printDetails(double outstanding)` olarak çıkarılır. Üye alanlar (`_orders`, `_name`) sorun çıkarmaz; döngüde hesaplanan geçici `outstanding` ise `getOutstanding()`'in dönüş değeri olur ve `printDetails`'e parametre olarak verilir. Sonuçta `printOwing()` iki satıra iner: `printBanner(); printDetails(getOutstanding());`.
- Bir metodu dörde bölmek akılda tutmayı zorlaştırır mı?
  - Hoca, insanın aynı anda sınırlı sayıda parçayla çalışabildiğini kabul etti; ama bir ayrıntı sorununda yalnız ilgili metotlara (`printDetails` ve `getOutstanding`) bakıldığını, diğerlerinin hiç açılmadığını söyledi.
  - Ders sonrası not: Hoca bu sınırı "5 artı eksi 2" diye verdi. Literatürde bilinen sınır büyük olasılıkla Miller'in "7 artı eksi 2" değeridir.
- Replace Temp with Query tartışması (ekrandan, 02:16:40 ve 02:20:00)
  - `getPrice()` sonrasında `basePrice()` iki kez çağrılır. `printReceipt()` örneğinde `item.getCharge()` döngü içinde iki kez çağrıldığı için 1000 öğede çağrı sayısı 1000'den 2000'e çıkar. Hocanın cevabı: asıl darboğaz büyük olasılıkla başka yerdedir; ölçmek için profiler kullanılır, kayıp varsa küçük adım geri alınır.
  - Haliç benzetmesi: köprüye gelen beş şerit köprüde dörde düşer; trafik ışıklarını düzenlemek yerine darboğazı (köprüyü) genişletmek akışı rahatlatır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.